

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Объектно-ориентированное программирование»

«К защите допустить»
Руководитель курсового проекта
ассистент кафедры информатики
_____ А.Г. Бокун
____.____.2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:
**«ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ГЕНЕРАЦИИ ИЗОБРАЖЕНИЙ
ФРАКТАЛЬНОГО ПЛАМЕНИ»**

БГУИР КП 6-05 0612 02 26 ПЗ

Выполнил студент группы 453501
СКРОБОТ Денис Алексеевич

(подпись студента)

Курсовой проект представлен на
проверку _____._____.2026

(подпись студента)

Минск 2026

СОДЕРЖАНИЕ

Введение	4
1 Анализ аналогов	5
1.1 Apophysis	5
1.2 JWildfire	6
1.3 Chaotica	7
1.4 Сравнительный анализ	8
2 Формирование функциональных требований	8
3 Анализ литературы	9
3.1 Принципы объектно-ориентированного проектирования	9
3.2 Принципы SOLID и качество кода	9
4 Проектирование системы	10
4.1 Архитектура микросервисов	10
4.2 Взаимодействие сервисов	10
4.3 Модель данных и хранение истории	10
4.4 Миграция от монолита к микросервисам	11
4.5 Сборка и запуск	11
4.6 Аутентификация пользователей	11
5 Технологии программирования, используемые для решения поставленных задач	11
5.1 Язык программирования Java	12
5.2 Фреймворк Spring	13
5.3 Система сборки Gradle	14
5.4 База данных PostgreSQL	15
5.5 Контейнеризация Docker и Docker Compose	15
5.6 Google OAuth 2.0 API	15
5.7 Интерфейс	16
5.8 Среда разработки Visual Studio Code	17
5.9 Хостинг и развёртывание приложения	18
6 Тестирование программного обеспечения	20
6.1 Стратегия тестирования	20
6.2 Сценарии тестирования	20
Заключение	21
Список использованных источников	22

ВВЕДЕНИЕ

Фрактальная графика относится к области визуализации сложных математических моделей и активно используется в цифровом искусстве, дизайне и научной визуализации. Одним из наиболее выразительных направлений является генерация фрактального «пламени», предложенная С. Дрейвсом, где изображение формируется итеративными нелинейными преобразованиями и последующей тональной обработкой плотности [1]. Благодаря богатству форм и высокой детализации такие изображения востребованы как в творческих проектах, так и в обучении математическому моделированию.

Современные инструменты генерации фракталов чаще представлены настольными приложениями и ориентированы на локальную работу пользователя. При этом растёт спрос на веб-сервисы, обеспечивающие доступ с разных устройств, хранение истории и возможность публикации результатов. Для решения этих задач требуется система, сочетающая удобный пользовательский интерфейс, вычислительно ёмкий рендеринг и надёжное хранение артефактов.

Целью курсового проекта является создание распределённой системы генерации фракталов с веб-интерфейсом, поддержкой аутентификации пользователей и хранением истории созданных работ. Архитектура системы построена на принципах микросервисов, что облегчает масштабирование и независимое развитие компонентов [2]. Разработка выполнена в соответствии с принципами объектно-ориентированного проектирования и ориентирована на практическое применение современных инструментов сборки и развёртывания.

Для достижения поставленной цели были решены следующие задачи:

- анализ существующих решений и выявление их сильных и слабых сторон;
- формирование функциональных и нефункциональных требований к системе;
- проектирование микросервисной архитектуры и модели данных;
- выбор технологий реализации, сборки и развёртывания;
- разработка и тестирование ключевых сценариев работы пользователей.

1 Анализ аналогов

Для определения требований и позиционирования проекта были рассмотрены наиболее известные решения в области генерации фрактальных пламен. Сравнение выполнено по критериям: доступность (desktop/web), качество рендеринга, удобство настройки параметров, наличие истории и механизмов публикации результатов.

1.1 Apophysis

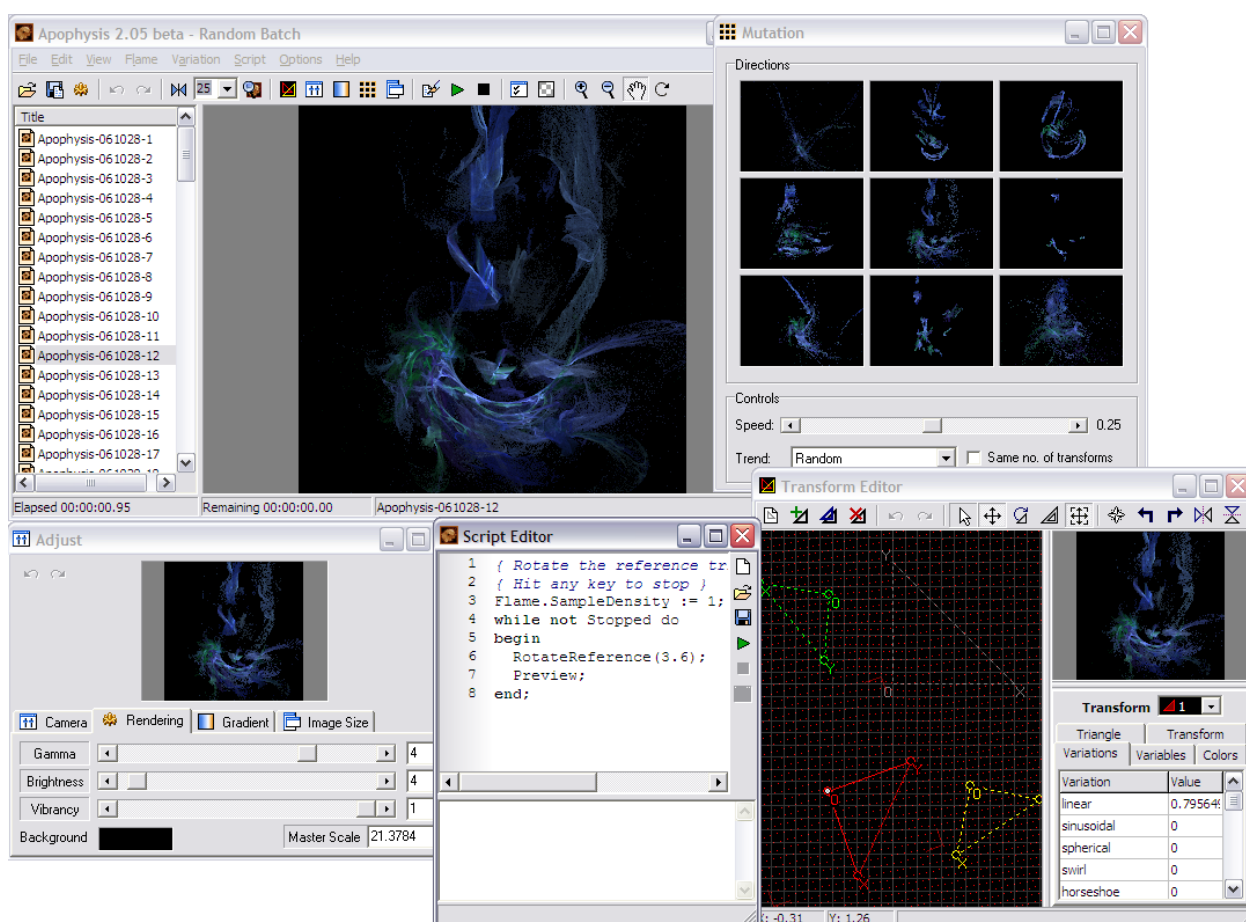


Рисунок 1 – Скриншот пользовательского интерфейса Apophysis с несколькими открытыми окнами

Apophysis — один из наиболее известных редакторов фрактальных пламен. Программа распространяется как open-source и предоставляет богатый набор инструментов для ручной настройки трансформаций, палитр и параметров рендера [3]. К достоинствам можно отнести простоту использования и обилие обучающих материалов. Основные ограничения связаны с настольным форматом, отсутствием встроенного хранения истории и невозможностью работы в распределённой среде.

Дополнительно стоит отметить, что основная работа в Apophysis строится вокруг ручного подбора вариаций и параметров, поэтому пользователь часто вынужден сохранять множество промежуточных файлов. Отсутствие центрального хранилища и единой учётной записи делает обмен результатами и совместную работу затруднительными.

1.2 JWildfire

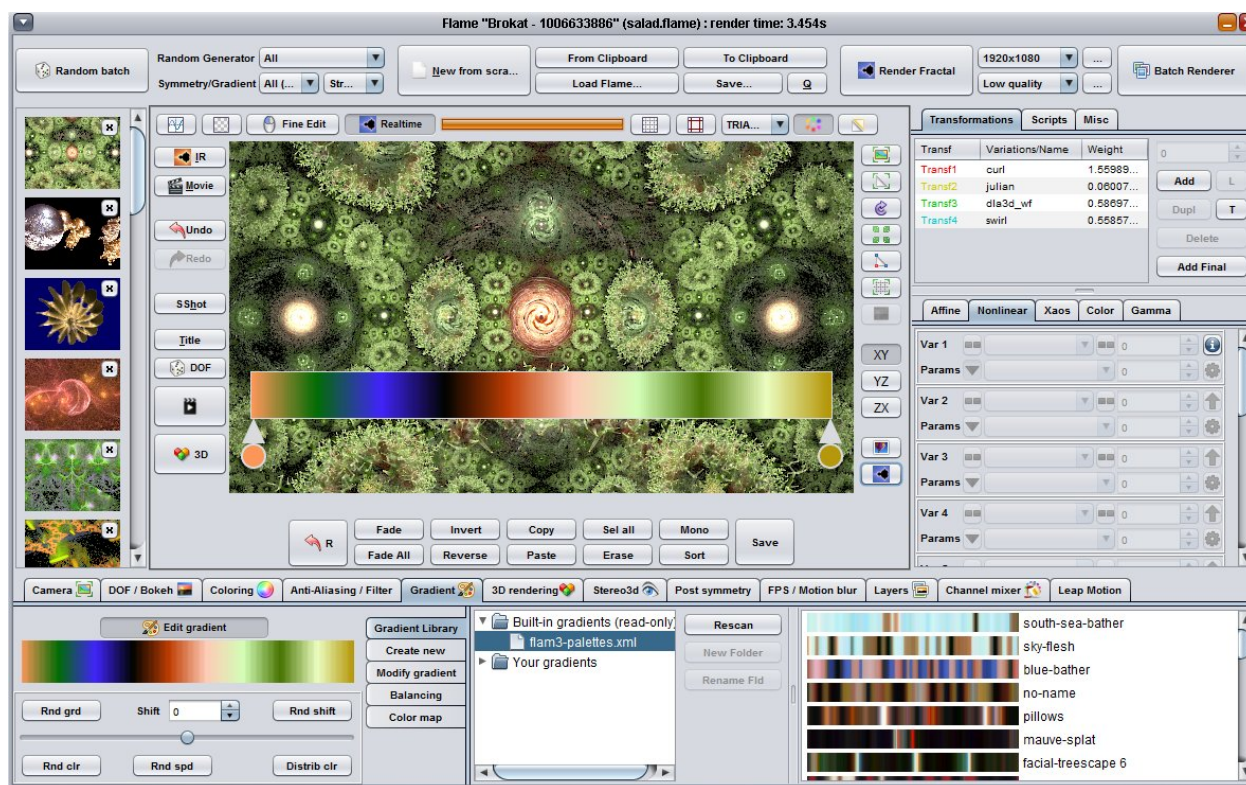


Рисунок 2 – Скриншот пользовательского интерфейса JWildfire

JWildfire — мощный кроссплатформенный редактор фрактальных пламен с большим набором вариаций и встроенными генераторами случайных фракталов [4]. Решение предоставляет широкие возможности, но требует значительного времени на освоение и ориентировано на опытных пользователей. Встроенные механизмы совместной работы и веб-публикации отсутствуют, а хранение результатов полностью локально.

Сильной стороной JWildfire является гибкость и глубина настроек, что делает его удобным для экспериментов и профессионального использования. Однако сложный интерфейс и необходимость локальной установки снижают доступность для начинающих пользователей и не решают задачу быстрого доступа к проектам с разных устройств.

1.3 Chaotica

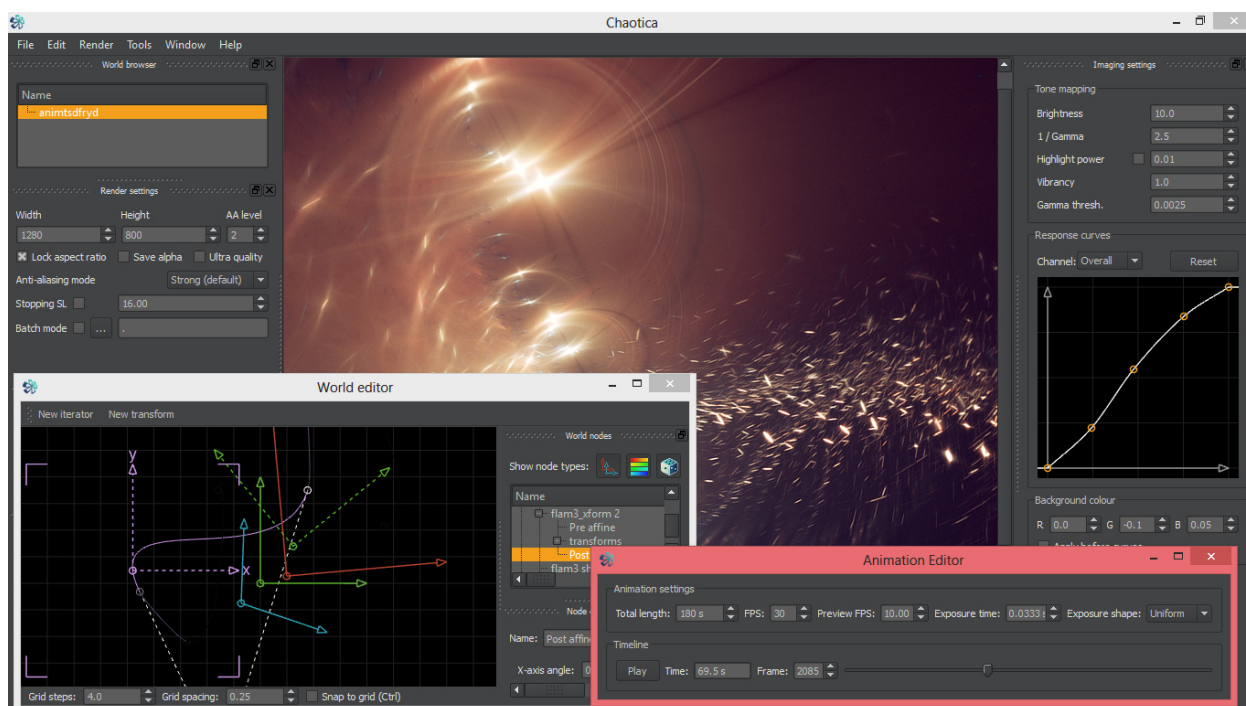


Рисунок 3 – Скриншот анимации фрактала, создаваемой в Chaotica

Chaotica — коммерческое приложение, ориентированное на получение высококачественных изображений и ускоренный рендеринг (включая GPU) [5]. Преимущества: высокая скорость и качество, развитый инструментарий анимации. Недостатки: закрытость, платная модель распространения и ориентация на локальную работу без интегрированного облачного хранения и пользовательских профилей.

В практической эксплуатации продукт удобен для студийных задач и финального качества рендера, однако ориентирован на автономное использование. Это ограничивает возможность модификаций, интеграции с веб-архивом и построения персонализированного профиля пользователя.

1.4 Сравнительный анализ

Таблица 1 – Сравнение аналогов по ключевым критериям

Продукт	Сильные стороны	Ограничения
Arphysics	Простота использования, open-source, богатая база примеров	Только desktop, нет истории и веб-доступа
JWildfire	Большое количество вариаций, кроссплатформенность, мощный набор функций	Высокая сложность интерфейса, локальное хранение, отсутствие веб-режима
Chaotica	Высокое качество рендера, поддержка GPU, развитая анимация	Платная лицензия, закрытость, отсутствие облачной истории

Анализ показал, что существующие решения преимущественно ориентированы на локальный рабочий процесс и не обеспечивают удобной работы с историей и публикацией результатов. Это определило требования к проектируемой системе, в частности необходимость веб-интерфейса, учётных записей и долговременного хранения рендеров.

Следовательно, ключевым конкурентным преимуществом будущей системы становится доступность из браузера и единое пространство проектов, где результаты сохраняются и доступны для повторного использования.

2 Формирование функциональных требований

На основе анализа аналогов сформированы функциональные требования к системе генерации фракталов:

- аутентификация пользователя через Google OAuth 2.0 и создание пользовательского профиля;
- создание нового проекта фрактала с возможностью выбора пресета или случайной генерации;
- редактирование параметров фрактала (вариации, аффинные преобразования);
- сохранение параметров и результата рендера в историю пользователя;

- экспорт результатов в популярных форматах изображений.
- Сформированы также нефункциональные требования:
- производительность: приемлемое время рендера при стандартных параметрах качества;
 - масштабируемость: возможность увеличения вычислительных ресурсов без изменения логики;
 - надёжность: сохранность истории и устойчивость к сбоям отдельных сервисов;
 - безопасность: защита пользовательских данных и корректная работа OAuth-потока;
 - переносимость: развёртывание системы в Docker-окружении;
 - сопровождаемость: модульная структура и единый процесс сборки через Gradle.

3 Анализ литературы

3.1 Принципы объектно-ориентированного проектирования

Объектно-ориентированный подход опирается на понятия инкапсуляции, наследования, полиморфизма и абстракции. Эти принципы позволяют моделировать предметную область в виде взаимосвязанных классов, обеспечивая повторное использование и расширяемость кода [6]. Для разрабатываемой системы это особенно важно, поскольку логика генерации фракталов включает множество разновидностей преобразований, параметров рендера и вариаций, которые удобно представлять в виде иерархий объектов.

В проекте объектные модели описывают параметры рендера, состояние проекта и пользовательский контекст, что позволяет разделить вычислительную и интерфейсную части. Это упрощает развитие функциональности и делает код более устойчивым к изменениям.

3.2 Принципы SOLID и качество кода

Принципы SOLID ориентированы на снижение связности и повышение устойчивости к изменениям. В частности, разделение ответственности и открытость к расширению позволяют добавлять новые алгоритмы генерации и модули без переработки существующего кода [7]. Эти принципы были использованы при проектировании сервисов, чтобы обеспечивать независимость компонентов и удобство тестирования.

На практике это выразилось в выделении отдельных интерфейсов для рендера, хранения и пользовательских операций, а также в уменьшении числа прямых зависимостей между слоями. Такой подход упростил модульное тестирование и локализацию ошибок.

4 Проектирование системы

4.1 Архитектура микросервисов

Система реализована в виде трёх микросервисов:

- сервис генерации фракталов — отвечает за вычисление предпросмотра и финального рендера;
- сервис хранения — отвечает за сохранение бинарных файлов и доступ к истории; внутренние детали сведены к общему описанию, поскольку разработка сервиса выполняется сторонним исполнителем;
- сервис веб-интерфейса и оркестрации — обеспечивает пользовательский интерфейс, оркестрацию запросов и хранение метаданных.

Такое разделение позволило выделить вычислительно тяжёлую часть в отдельный сервис и облегчить масштабирование, а также упростить независимое развитие компонентов.

Кроме того, веб-сервис стал единой точкой входа для пользовательских сценариев, а вычислительный контур может масштабироваться горизонтально под нагрузку рендеринга.

4.2 Взаимодействие сервисов

Взаимодействие между сервисами реализовано через REST API с передачей данных в формате JSON. Сервис веб-интерфейса инициирует генерацию, передаёт параметры в сервис рендера и получает идентификатор задачи. После завершения рендера результат отправляется в сервис хранения, а метаданные фиксируются в базе данных сервиса веб-интерфейса. Такой подход обеспечивает чёткое разделение ответственности и позволяет масштабировать вычислительный контур независимо от пользовательского интерфейса.

Коммуникация строится по принципу кратких запросов и асинхронных задач: сервис рендера возвращает идентификатор, а веб-сервис периодически запрашивает статус и обновляет интерфейс. В случае ошибок используются единые коды и сообщения, что упрощает диагностику.

4.3 Модель данных и хранение истории

В системе выделены следующие основные сущности: пользователь, проект фрактала, рендер, набор параметров и файлы результата. История представляет собой набор версий проекта с привязкой к итоговому изображению и параметрам рендера. Хранение изображений организовано в специализированном сервисе хранения, а метаданные сохраняются в базе данных сервиса веб-интерфейса. Это обеспечивает быстрый доступ к истории и возможность восстановления состояния проекта.

Для ускорения поиска применяются индексация по пользователю и дате, а версия проекта хранится вместе с набором параметров, что позволяет воспроизвести результат в любой момент.

4.4 Миграция от монолита к микросервисам

Исходный прототип представлял собой монолитное приложение. В ходе работы была выполнена декомпозиция: вычислительная логика рендера вынесена в отдельный сервис, хранение файлов выделено в специализированный сервис, а основной сервис веб-интерфейса и оркестрации превратился в оркестратор и точку входа для клиентского интерфейса.

Миграция выполнялась поэтапно: сначала были выделены границы домена, затем определены контракты API и только после этого осуществлён перенос вычислительной логики в отдельный сервис.

4.5 Сборка и запуск

Сборка сервисов осуществляется в едином Gradle-проекте, что обеспечивает согласованность зависимостей и единый набор задач сборки [8]. Для запуска используется Docker: каждый сервис упакован в отдельный контейнер, а совместный запуск выполняется через Docker Compose [9].

В результате появилась единая команда сборки и единый сценарий запуска, что облегчает локальную разработку и автоматизированное тестирование.

4.6 Аутентификация пользователей

Для входа пользователей применяется Google OAuth 2.0. Клиентский интерфейс инициирует авторизацию, после чего сервис веб-интерфейса обменивает код на токен и получает базовую информацию о пользователе [10]. Процесс соответствует стандартному протоколу OAuth 2.0 и позволяет не хранить пароли в системе [11].

Подробное обоснование выбранных технологий и инструментов приведено в следующем разделе.

5 Технологии программирования, используемые для решения поставленных задач

Здесь мы рассмотрим, какие технологии программирования были выбраны и по какой причине выбор был именно таким.

5.1 Язык программирования Java



Рисунок 4 – Java

Язык программирования Java является объектно-ориентированным языком, разработанным компанией Sun Microsystems и в настоящее время поддерживаемым корпорацией Oracle. Java обеспечивает высокий уровень абстракции, платформенную независимость и безопасность, что делает его одним из самых популярных языков в мире.

Одним из основных преимуществ Java является его независимость от платформы, достигаемая благодаря использованию виртуальной машины Java (JVM), которая исполняет байт-код, одинаковый для всех операционных систем. Это свойство позволяет создавать переносимые приложения без необходимости модификации исходного кода.

Java поддерживает основные принципы объектно-ориентированного программирования, такие как наследование, инкапсуляция, полиморфизм и абстракция. Эти особенности обеспечивают высокую модульность и повторное использование кода. [12]

В рамках разработки проекта язык Java был выбран как основной инструмент программирования благодаря следующим преимуществам:

- наличие обширной стандартной библиотеки, включающей средства для сетевого взаимодействия, работы с потоками, графическим интерфейсом и структурами данных;
- развитая экосистема, включающая библиотеки и фреймворки, ускоряющие процесс разработки;
- встроенная система управления памятью с автоматическим сборщиком мусора, упрощающая разработку и снижает вероятность ошибок, связанных с управлением ресурсами;
- высокая читаемость и строгая типизация, что способствует написанию надёжного и легко сопровождаемого кода.

В процессе разработки проекта с использованием Java были реализованы следующие компоненты: взаимодействие с серверной частью

через RESTful API, обработка пользовательских действий и реализация механизма обновления состояния в реальном времени.

Таким образом, использование языка Java в проекте позволило обеспечить кроссплатформенность, надёжность и структурированность разрабатываемого программного продукта.

Дополнительно использовались стандартные средства работы с коллекциями и многопоточностью, что дало возможность эффективно управлять очередью задач рендера и обменом данными между компонентами. Это снизило задержки при обработке запросов и повысило устойчивость сервиса к пиковым нагрузкам.

5.2 Фреймворк Spring



Рисунок 5 – Spring Framework

Spring — это мощный и гибкий фреймворк с открытым исходным кодом для разработки корпоративных и веб-приложений на языке Java. Благодаря своей модульной структуре, высокой производительности и удобству масштабирования, Spring широко применяется при создании распределённых и высоконагруженных систем, включая веб-сервисы и многопользовательские онлайн-платформы.

Одной из ключевых особенностей Spring является внедрение зависимостей (Dependency Injection), которое способствует слабой связанности компонентов приложения и облегчает их тестирование, повторное использование и расширение. Фреймворк построен по принципу инверсии управления (Inversion of Control), где управление жизненным циклом объектов осуществляется специальным контейнером.

В проекте фреймворк Spring используется для реализации серверной части приложения, обеспечивающей работу с пользователями, обработку запросов, взаимодействие с базой данных и реализацию защитных механизмов.

В пользовательской части предусмотрены рейтинги и элементы соревновательности в сообществе авторов, поэтому в API учитываются действия участников и их вклад в общую активность.

В процессе разработки были использованы следующие модули Spring:

– Spring Boot — модуль, предназначенный для упрощения конфигурации и быстрой инициализации проекта. Он предоставляет встроенный сервер и автоматическую настройку компонентов, что позволило сосредоточиться на реализации логики приложения;

– Spring Web — модуль, реализующий обработку HTTP-запросов, маршрутизацию, и построение RESTful API. С его помощью реализовано взаимодействие клиентской и серверной частей, включая передачу данных о пикселях, рейтингах и действиях игроков;

– Spring Data — модуль для работы с базой данных с использованием объектно-реляционного отображения. Он позволяет создавать репозитории и взаимодействовать с таблицами базы данных через Java-классы, что упрощает реализацию слоя доступа к данным;

– Spring Security — модуль, обеспечивающий защиту приложения. В проекте он используется для реализации регистрации, аутентификации и авторизации пользователей, а также ограничения доступа к защищённым ресурсам в зависимости от роли.

Использование Spring позволило добиться высокой скорости обработки запросов, надёжности и масштабируемости серверной части. Благодаря модульной архитектуре фреймворка, разработка велась по принципу разделения ответственности, что улучшило структуру проекта и упростило поддержку и развитие функционала. [13]

Дополнительно применялись встроенные механизмы валидации данных и централизованная обработка ошибок, что обеспечило единый формат ответов API и предсказуемость поведения системы.

5.3 Система сборки Gradle

Gradle использовался как единый инструмент сборки для всех модулей проекта. Он обеспечил многомодульную структуру, централизованное управление версиями зависимостей и стандартные задачи для сборки, тестирования и упаковки. Благодаря единым скриптам удалось синхронизировать процесс разработки и исключить расхождения между сервисами. [8]

5.4 База данных PostgreSQL



Рисунок 6 – PostgreSQL

Для хранения данных используется система управления базами данных PostgreSQL. Это современное, высокопроизводительное и надёжное решение, широко применяемое для создания отказоустойчивых и масштабируемых серверных приложений.

Взаимодействие приложения с PostgreSQL реализовано через Spring Data JPA, что обеспечивает надёжный и гибкий доступ к данным без необходимости явного написания SQL-запросов.

Выбор PostgreSQL обусловлен её устойчивостью к сбоям, поддержкой транзакций, возможностями масштабирования и глубокой интеграцией с инструментами Java и Spring.

Структура данных была организована с использованием нормализованных таблиц, индексов по ключевым полям и ограничений целостности. Это позволило обеспечить корректность истории проектов и ускорить выборки по пользователю и времени.

5.5 Контейнеризация Docker и Docker Compose

Для развёртывания использовалась контейнеризация Docker. Каждый микросервис упакован в отдельный образ, а общий сценарий запуска описан в Docker Compose, где заданы сети, переменные окружения и зависимости между сервисами. Это обеспечило воспроизводимость окружения и упростило перенос системы между локальной разработкой и сервером. [9]

5.6 Google OAuth 2.0 API

Для реализации аутентификации пользователей в приложении использовалась технология Google OAuth 2.0 API, интегрированная с помощью фреймворка Spring Security. OAuth 2.0 представляет собой

широко используемый протокол авторизации, позволяющий сторонним приложениям безопасно получать ограниченный доступ к пользовательским данным без необходимости запрашивать пароль.

С помощью данной технологии пользователи получают возможность входа в систему через свою учётную запись Google. Это повышает уровень безопасности приложения и упрощает процесс аутентификации, поскольку пользователь не вводит логин и пароль вручную.

После успешной авторизации Google перенаправляет пользователя обратно в приложение с временным авторизационным кодом. Этот код обменивается на access token, после чего приложение может получить базовую информацию о пользователе.

Использование Google OAuth 2.0 API в сочетании со Spring Security позволило реализовать следующие преимущества:

- безопасная авторизация без хранения паролей пользователей;
- удобство и быстрота входа в систему;
- снижение барьера входа для новых пользователей.

Дополнительно был реализован контроль запрашиваемых прав доступа (scopes) и безопасная обработка callback-адресов, что исключает несанкционированные перенаправления. При необходимости используется повторное получение токена без повторного входа пользователя.

5.7 Интерфейс

Во внешнем интерфейсе приложения использовались стандартные веб-технологии — HTML, CSS и JavaScript [14], а также шаблонизатор Thymeleaf, встроенный в стек Spring Boot.

HTML (HyperText Markup Language) применялся для создания структуры веб-страниц и определения размещения элементов пользовательского интерфейса. Все страницы проекта были реализованы с учётом семантической разметки, что облегчает их поддержку и улучшает восприятие контента пользователями.

CSS (Cascading Style Sheets) использовался для стилизации HTML-элементов и создания визуально привлекательного интерфейса. Основное внимание уделялось адаптивной верстке и обеспечению читаемости интерфейса на различных устройствах.

JavaScript применялся для реализации интерактивности на клиентской стороне. С его помощью реализованы такие функции, как динамическое обновление содержимого без перезагрузки страницы (через асинхронные запросы), обработка событий пользователя и взаимодействие с серверной частью приложения через REST API.

Особое внимание в проекте уделялось интеграции с Thymeleaf — серверным Java-шаблонизатором, тесно интегрированным с Spring Boot. С его помощью осуществлялся рендеринг HTML-страниц с динамическими

данными, передаваемыми с сервера. Использование Thymeleaf позволило избежать избыточной ручной генерации HTML и упростить передачу данных от сервера к клиенту, благодаря поддержке выражений и условной логики в шаблонах.

Интерфейс был спроектирован с учётом адаптивности, чтобы пользователь мог работать как с настольного ПК, так и с мобильных устройств. Основные сценарии включают создание проекта, настройку параметров и просмотр истории с быстрым доступом к сохранённым работам.

5.8 Среда разработки Visual Studio Code

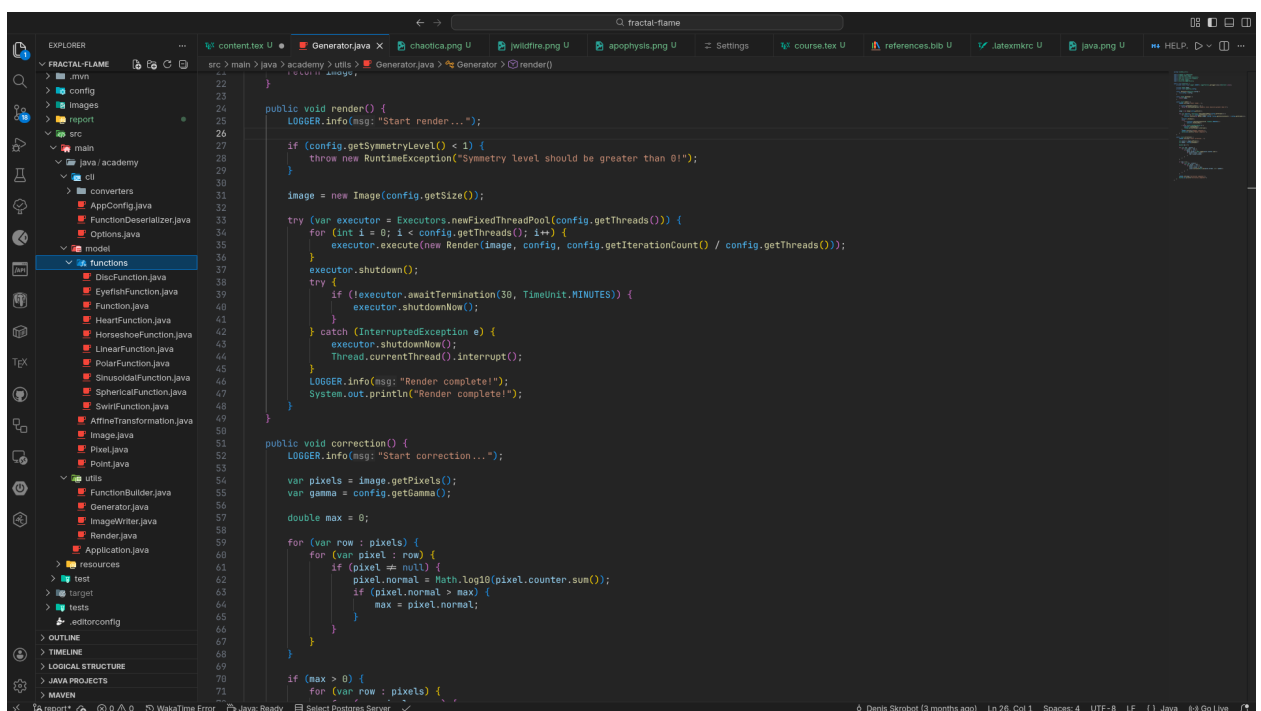


Рисунок 7 – Экран редактирования кода Visual Studio Code

Visual Studio Code (VS Code) — это лёгкая, но мощная интегрированная среда разработки, разработанная компанией Microsoft. С момента своего выхода в 2015 году она завоевала огромную популярность благодаря простоте, скорости и гибкости. VS Code является кроссплатформенным инструментом, доступным на Windows, macOS и Linux, что делает его универсальным решением для разработчиков по всему миру. [15]

VS Code не перегружена лишними функциями, но в то же время предоставляет всё необходимое для разработки: интеллектуальную подсветку синтаксиса, автодополнение, отладку, поддержку Git, систему контроля версий и встроенный терминал. Это позволяет разрабатывать и тестировать код быстро и эффективно, без лишних задержек.

Основные причины выбора VS Code включают:

1. Лёгкость и быстроедействие. VS Code быстро запускается и работает с минимальными задержками, что делает её идеальной для разработки фронтенд-части и быстрого прототипирования.
2. Расширяемость. Благодаря множеству доступных расширений, VS Code легко адаптируется под любые задачи.
3. Интеграция с Git и другими системами контроля версий. VS Code имеет встроенную поддержку Git, что упрощает работу с репозиториями и позволяет эффективно управлять версиями проекта.

Разработка всех компонентов проекта выполнялась исключительно в Visual Studio Code, что обеспечило единообразие среды и повторяемость процесса. Дополнительно использовались расширения для Java, Docker и работы с L^AT_EX, что позволило вести и разработку, и оформление документации в одном инструменте. Даже этот документ написан в VS Code с использованием L^AT_EX.

5.9 Хостинг и развёртывание приложения

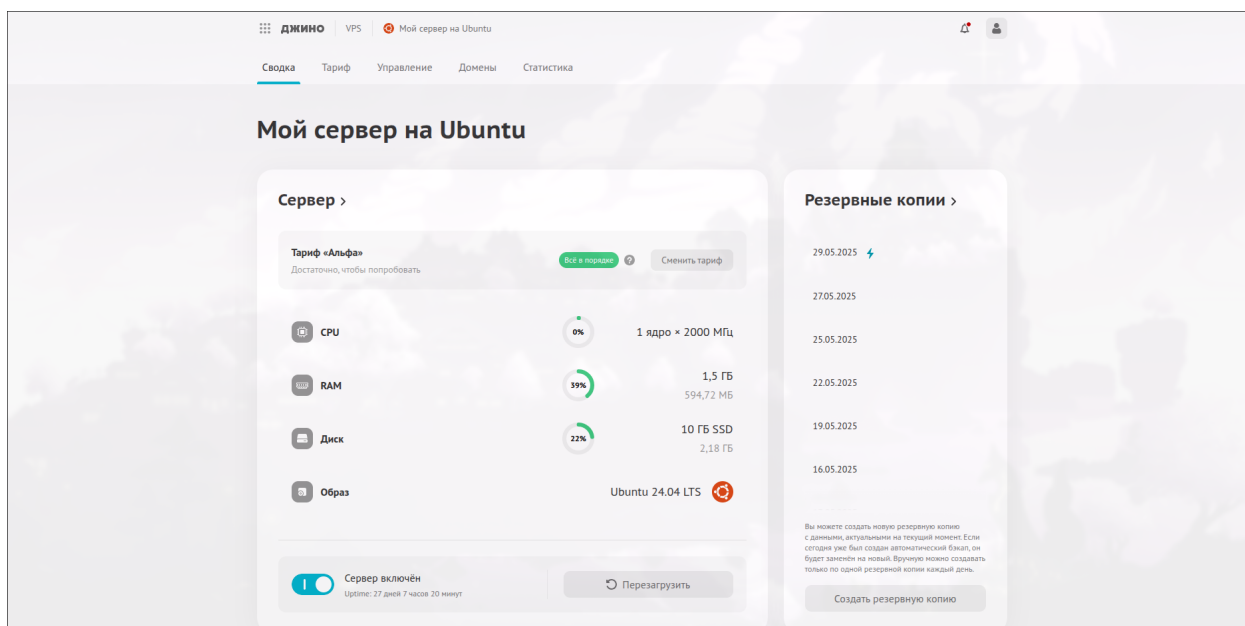


Рисунок 8 – Страница сервера на jino.ru

Финальная версия системы разворачивается в Docker-контейнерах на VPS; ниже приведён общий сценарий размещения и администрирования.

Для размещения серверной части проекта использовался платный хостинг-провайдер Jino.ru. Выбор данного провайдера был обусловлен возможностью полного управления сервером через SSH-доступ, поддержкой системных сервисов, а также интеграцией доменов и SSL-сертификатов. [16]

Развёртывание приложения осуществляется на выделенный виртуальный сервер (VPS), работающий под управлением операционной системы Linux. Доступ к серверу выполняется с использованием безопасного протокола SSH, что позволяет выполнять все действия удалённо: загрузку файлов, настройку служб и управление серверной частью.

В составе развёртывания применяется обратный прокси для маршрутизации запросов и разграничения доступа к сервисам, а также централизованное логирование. Это упростило администрирование и позволило быстро выявлять ошибки в работе компонентов.

Refactoring required

Сборка проекта осуществляется с использованием системы Gradle. После успешной сборки на локальной машине создаётся JAR-файл, который загружается на сервер с помощью SCP. Развёртывание приложения производится вручную, без использования CI/CD, с контролем всех этапов.

Для автоматического запуска и надёжной поддержки работоспособности серверной части был создан отдельный systemd-сервис, зарегистрированный в системе как юнит. Этот сервис обеспечивает:

- автоматический запуск приложения при загрузке сервера;
- перезапуск в случае сбоев;
- логирование через системный журнал.

Конфигурация systemd-юнита включает в себя путь к исполняемому JAR-файлу, указание рабочей директории, параметров Java-машины и стандартных параметров управления службой.

Для доступа к приложению по HTTPS был зарегистрирован собственный домен, приобретённый через панель управления Jino. К домену подключён wildcard-сертификат, позволяющий защищать не только основной домен, но и все его поддомены. Сертификат установлен на стороне веб-сервера, который при необходимости может проксировать запросы к приложению, обеспечивая безопасную и надёжную маршрутизацию трафика.

Использование Jino в качестве хостинг-платформы позволило обеспечить:

- полную автономность и контроль над серверной средой;
- надёжную работу приложения в режиме 24/7;
- безопасный доступ пользователей по защищённому протоколу HTTPS;
- удобную интеграцию домена и сертификатов.

Развёртывание проекта в реальной среде подтвердило его работоспособность вне среды разработки и позволило провести полноценное тестирование функциональности в условиях, приближенных к промышленной эксплуатации.

6 Тестирование программного обеспечения

6.1 Стратегия тестирования

Тестирование системы проводилось на нескольких уровнях: модульном, интеграционном и системном. На модульном уровне проверялась корректность отдельных компонентов генерации и обработки параметров. Интеграционные тесты подтверждали корректность взаимодействия микросервисов, а системные тесты моделировали реальную работу пользователя. Подход соответствует классическим рекомендациям по организации тестирования программного обеспечения [17].

Для проверки критических сценариев использовались контрольные наборы параметров рендера и заранее подготовленные эталонные результаты. Это позволило выявлять регрессии при изменениях алгоритмов и конфигурации.

6.2 Сценарии тестирования

Ниже приведены основные сценарии, подтверждающие работоспособность системы:

1. Авторизация пользователя через Google OAuth: успешный вход, создание профиля, корректное завершение сессии.
2. Создание нового проекта фрактала: генерация случайного пресета и отображение предпросмотра.
3. Редактирование параметров: изменение вариаций и палитры с мгновенным обновлением предпросмотра.
4. Запуск финального рендера: создание асинхронной задачи и корректное отображение её статуса.
5. Сохранение результата: запись изображения и параметров в историю пользователя.
6. Повторное открытие проекта: восстановление параметров и повторный рендер.
7. Экспорт изображения: загрузка файла в формате PNG и проверка его целостности.
8. Ограничение доступа: проверка недоступности частных работ для сторонних пользователей.

Результаты тестирования подтвердили корректность работы ключевых функций и соответствие заявленным требованиям.

Отдельно фиксировались временные показатели рендера и устойчивость к отказам отдельных сервисов, что подтвердило работоспособность системы в условиях повышенной нагрузки.

ЗАКЛЮЧЕНИЕ

Refactoring required

В ходе выполнения курсового проекта была спроектирована и реализована система генерации фракталов в виде микросервисной архитектуры. Проведён анализ аналогов, сформированы требования, описаны архитектурные решения и технологический стек. Реализована поддержка аутентификации пользователей, хранение истории работ и контейнеризированное развёртывание.

Полученный результат демонстрирует, что выбранный подход обеспечивает удобство использования, масштабируемость и надёжность. Итоги работы могут быть использованы в дальнейшем развитии проекта, включая расширение алгоритмов генерации, внедрение дополнительных инструментов публикации и совершенствование пользовательского интерфейса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Draves, S. The Fractal Flame Algorithm / S. Draves, E. Reckase // The Flame Algorithm. — 2003. — 41 P.
- [2] Newman, Sam. Building Microservices / Sam Newman. — 2 edition. — O'Reilly Media, 2021.
- [3] Apophysis: Fractal Flame Editor [Электронный ресурс]. — Режим доступа : <https://sourceforge.net/projects/apophysis/>.
- [4] JWildfire: Fractal Flame Generator (Repository) [Электронный ресурс]. — Режим доступа : <https://github.com/thargor6/JWildfire>.
- [5] Chaotica Fractal Flame Software [Электронный ресурс]. — Режим доступа : <https://www.chaoticafractals.com/>.
- [6] Booch, G. Object-Oriented Analysis and Design with Applications / G. Booch. — 3 edition. — Addison-Wesley Professional, 2007.
- [7] Martin, Robert C. Agile Software Development: Principles, Patterns, and Practices / Robert C. Martin. — Prentice Hall, 2002.
- [8] Gradle User Manual [Электронный ресурс]. — Режим доступа : <https://docs.gradle.org/current/userguide/userguide.html>.
- [9] Docker Compose Documentation [Электронный ресурс]. — Режим доступа : <https://docs.docker.com/compose/>.
- [10] Using OAuth 2.0 to Access Google APIs [Электронный ресурс]. — Режим доступа : <https://developers.google.com/identity/protocols/oauth2>.
- [11] Hardt, D. The OAuth 2.0 Authorization Framework: Tech. Rep. RFC 6749 / D. Hardt: IETF, 2012. — Режим доступа : <https://www.rfc-editor.org/rfc/rfc6749>.
- [12] Schildt, H. Java: The Complete Reference, Ninth Edition / H. Schildt. — McGraw-Hill Education, 2014. — Pp. 17 – 23.
- [13] Walls, Craig. Spring in Action / Craig Walls. — 6 edition. — Manning Publications, 2022.
- [14] Robbins, Jennifer Niederst. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics / Jennifer Niederst Robbins. — 5 edition. — O'Reilly Media, 2018.
- [15] Visual Studio Code Documentation [Электронный ресурс]. — Режим доступа : <https://code.visualstudio.com/docs>.
- [16] Jino.ru: виртуальный хостинг и VPS [Электронный ресурс]. — Режим доступа : <https://jino.ru/>.
- [17] Myers, Glenford J. The Art of Software Testing / Glenford J. Myers, Corey Sandler, Tom Badgett. — 3 edition. — Wiley, 2011.
- [18] Design Patterns: Elements of Reusable Object-Oriented Software / E. Gamma, R. Helm, R. Johnson, J. Vlissides. — Addison-Wesley, 1994.
- [19] Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software / Eric Evans. — Addison-Wesley, 2003.